Fundamentals

Verification & Testing

How to review AI output, write effective tests, and catch bugs before they compound.

Testing and the type system are your most powerful verification tools in an AI-enabled interview. Tests make AI-generated code better because the model has concrete targets to hit. They make verification faster because you run them instead of reading every line. And they show the interviewer you care about correctness, which is one of the top evaluation criteria across every company we've talked to. Types do a lot of the same work, catching a whole class of bugs before you ever run the code.

How you approach testing depends on the format.

Test-driven development in open-ended interviews

In **open-ended interviews** where you're building from scratch, write tests before you start implementing. This is **test-driven development** (TDD), and while not every team practices it, it becomes especially powerful when you're working with AI. This might feel counterintuitive when the clock is ticking, but it's one of the highest-leverage things you can do.

> TDD isn't just an interview trick. This same workflow, writing tests first and using them to guide AI-generated code, is how many engineers are working day-to-day now. Building this habit for your interview will carry directly into your actual job.

Start by agreeing on the test cases with your interviewer. Walk them through the inputs and expected outputs you're planning to test against, and make sure you're both aligned on what "correct" looks like before you write any implementation code. This conversation often surfaces misunderstandings about the requirements that would have cost you significant time to discover later through debugging. It also gives you a shared definition of "done" for each phase, so when the tests pass, you can both confidently agree that the implementation is working and move on to the next step.

Once you have agreed-upon tests, the AI produces significantly better code because it has clear targets to hit. A prompt like "implement this function so that these test cases pass" gives the

model much more to work with than "implement this function." The tests constrain the output in exactly the right ways, reducing the chance of the AI drifting into a different interpretation of the problem.

Your verification workflow also becomes much simpler. Instead of reading through every line of AI-generated code to check correctness, you run the tests. If they pass, the code does what you agreed it should do. Your main job shifts from verifying the implementation to verifying the tests themselves, which is a much smaller surface area.

> After writing your core test cases, prompt the AI to generate additional tests for corner cases. Empty input, single element, duplicates, overflow, invalid input. Review these generated tests carefully though. AI-generated tests can have the same blind spots as AI-generated code: wrong expected values, missing edge cases, or tests that pass trivially because they don't actually exercise the logic.

What makes a good test

Whether you're writing tests yourself or reviewing AI-generated ones, a good test suite covers these categories:

1. **Happy path** - the standard case that should obviously work
2. **Empty/zero input** - empty arrays, empty strings, zero values, null
3. **Single element** - one item in a collection, one character in a string
4. **Boundary values** - min/max integers, first/last index, exactly at a limit
5. **Duplicates** - repeated values, duplicate keys, identical objects
6. **Invalid or unexpected input** - negative numbers, wrong types, malformed data
7. **Scale** - if the problem has a performance component, a test with larger input to confirm it doesn't time out

You don't need all of these for every function, but scanning this list before moving on helps you catch the gaps that are most likely to bite you later.

Adding tests in structured interviews

Structured interviews often come with provided test cases, but don't assume these are comprehensive. They usually cover the happy path and maybe one or two basic edge cases, but they're rarely exhaustive. The provided tests are there to get you started, not to fully validate your solution.

It's worth adding your own test cases for the corner cases and boundary conditions that the provided tests don't cover. If the problem involves a grid, what happens with a 1x1 grid? If it involves string matching, what about empty strings or single characters? These take seconds to write and they can catch the kinds of subtle bugs that would otherwise cost you ten minutes of confused debugging later in the interview.

This becomes especially valuable before the optimization phase. When you're changing the underlying algorithm, a solid test suite is your safety net. Run the full suite before you start optimizing to confirm everything passes, then run it again after each change to catch regressions immediately. Without this, you might "optimize" the solution and break something that was already working without realizing it until much later.

The type system is also a test suite

In statically typed languages, keeping your code compiling with clean types is a form of incremental verification that's easy to underuse in interviews. Every type error the compiler catches is a bug you didn't have to find by running code or reading through a stack trace.

After each generation, compile and run the type checker. Type errors are concrete and actionable. Hand them directly to the AI: "Fix the type error on line 42" is a precise prompt that produces better results than "something seems wrong here." Fixing types early also prevents cascading failures where a mismatch in one file breaks several others right before the optimization phase.

Watch out for the AI taking shortcuts with the type system. In TypeScript, it'll sometimes use `any` to silence a type error rather than fixing the underlying issue. In other languages, it might add unnecessary casts or ignore nullability. Treat these the same way you'd treat a deleted failing test. It's a correctness shortcut that needs to be fixed, not accepted. An interviewer who notices `any` sprinkled through your codebase will ask about it.

Type hygiene matters across languages. Go's interface system, Java's generics, Python's type annotations with mypy. Each one gives you a way to verify correctness before running a single

test. Use them.

What to test and when

Run your tests and compile/typecheck after each phase or meaningful milestone, not at the very end. If your plan has three steps, run the suite after completing each one. Candidates who wait until the end to see if everything works together often discover cascading failures with no time left to fix them. Testing incrementally means each failure is isolated to the most recent change, which makes it dramatically faster to diagnose and fix.

When a test does fail, form a hypothesis before prompting the AI to fix it. "I think this fails because the loop doesn't handle the case where the input array is empty" is a much more productive starting point than pasting the error and asking the AI to fix it. It keeps you in control of the debugging process and shows the interviewer you're reasoning about the code rather than delegating the thinking.

One useful move here: once you have your hypothesis, fire off an AI prompt with it at the same time you start investigating manually. "I'm going to look at this myself, but let me also ask the AI." You're not handing over the thinking. You've already formed the hypothesis. You're just running two parallel investigations. Whichever one finds the root cause first wins.

> Don't skip testing because you're worried about time. Candidates who skip tests consistently run into more debugging issues in the final third of the interview, which costs far more time than writing the tests would have. Testing is an investment that pays for itself almost immediately.